

Lab: Introduction to Solidity and Remix IDE

Objectives:

- To understand the Solidity programming language and the Ethereum Virtual Machine (EVM).
- To learn contract structures, data types, visibility, modifiers, and error handling.
- To build a comprehensive backend Smart Contract for a Decentralized Application (DApp) using Remix IDE.

Submission:

- Students should finish the checkpoints during the lab class.

Description:

In this comprehensive lab, you will learn the fundamentals of the Solidity programming language and write your own smart contracts for the Ethereum blockchain. You will write, test, and debug your code using the **Remix IDE**. Remix is widely regarded as the industry-standard environment for blockchain developers, offering an intuitive Graphical User Interface (GUI), debugging tools, and a rich ecosystem of plugins.

What is Solidity?

Solidity is a statically typed, object-oriented programming language designed specifically to run on the Ethereum Virtual Machine (EVM). It is the *de facto* standard for developing smart contracts across various blockchain platforms.

The language is highly influenced by C++, Python, and JavaScript; therefore, if you have prior experience with Java or JavaScript, you will find the syntax quite familiar. However, because executing code on the blockchain costs real money (gas), Solidity is highly computation-sensitive. It introduces several blockchain-specific paradigms that make it uniquely different from traditional programming languages, including:

- Strictly sized integers (**uint** and **int** ranging from 8 to 256 bits).
- Cryptographic data types like **address** and **mapping**.
- **events** for logging data to the blockchain.
- Specific function **modifiers** and state-reverting error-handling techniques.

Lab Focus

Throughout this session, you will explore the advanced features of Solidity. We will cover reference-typed data, error handling, detailed function keywords, and smart contract design patterns. All of this will culminate in solving a real-world problem by building the backend for a Decentralized Application (DApp).

Prerequisites:

- Familiarity with programming in at least one programming language
- Blockchain
- Web Browser. Recommended - Google Chrome or Firefox

Section 1: Core Concepts of Ethereum & EVM

Before writing code, it is crucial to understand the environment in which your smart contracts will live and execute.

1. The Ethereum Virtual Machine (EVM)

The EVM is the runtime environment for smart contracts in Ethereum. It is entirely **sandboxed and completely isolated**. This means that code running inside the EVM has no access to the broader network, the local filesystem, or other running processes. Even smart contracts have restricted, explicitly defined access to other smart contracts.

2. Ethereum Accounts

Every account in Ethereum possesses a balance in Ether (specifically measured in "**wei**", where **1 Ether = 10^{18} wei**) and a persistent key-value store called *storage* (mapping 256-bit words to 256-bit words). Ethereum features two types of accounts that share the same address space:

- **External Accounts:** Controlled by public-private key pairs (human users). The address is derived from the public key.
- **Contract Accounts:** Controlled by the executable code stored alongside the account details. The address is determined at the time the contract is created.
- *Note:* Regardless of whether an account stores code, the EVM treats both types equally.

3. Transactions

A transaction is a message sent from one account to another, which may include binary data (the payload) and Ether. There are three primary transaction scenarios:

- **Target is an External Account:** The Ether value of the transaction is simply transferred to the target.
- **Target is a Contract Account:** The target's code is executed, and the transaction's payload is provided as input data (parameters for a specific function).
- **Target is Empty:** If the transaction does not have a recipient, it creates a **new contract**. The output data of this execution is permanently stored as the code of the new contract.

4. Gas & Execution Costs

Because executing code utilizes network resources, transactions are charged a fee called **Gas**.

- The creator of a transaction sets a **gas_price** and must pay **gas_price * gas** upfront.
- As the EVM executes the transaction, the gas gradually depletes.
- If any gas remains after execution, it is refunded. However, if the gas runs out before completion, an **out-of-gas exception** is triggered, and *all* modifications made to the state are immediately reverted.

5. EVM Data Areas

The EVM manages data across four distinct areas. Understanding these is critical for writing gas-efficient Solidity code. When dealing with complex reference types (like arrays or strings), you must explicitly define which of these areas to use:

- **Storage:** Each account has its own persistent data area, saved permanently on the blockchain. It is highly expensive to read and even more costly to modify. Modifying a storage reference modifies the actual state variable.
- **Memory:** This is volatile data. A contract obtains a freshly cleared instance of memory for each message call. It creates a temporary copy of data that can be modified within the function. It is cheaper than storage, but the cost scales up as the memory size expands.
- **Calldata:** This is a special, non-persistent, read-only data location that contains the incoming function arguments. It consumes the lowest amount of gas. Always use **calldata** for arrays/strings passed into external functions when you do not need to modify them.
- **Stack:** The EVM operates as a stack machine, performing computations on a stack with a maximum size of 1024 elements. As a developer, you do not use a **stack** keyword; Solidity automatically uses the stack to hold simple, temporary value types (like **uint256** or **bool**) during function execution.

6. Advanced Contract Interactions

- **Message Calls & Delegatecall:** Contracts can interact with others via message calls. A special variant, **delegatecall**, executes the target address's code *within the context of the calling contract*. This means the storage, address, and balance remain those of the caller, enabling the creation of reusable **Libraries**.
- **Logs & Events:** The EVM can store data in specially indexed structures called *logs*. Solidity uses these to implement **Events**. While contracts cannot read log data after creation, DApps (via JavaScript) can efficiently listen for these events to update user interfaces.

- **Contract Creation (Create):** Contracts can dynamically generate other contracts using a special opcode, returning the address of the newly spawned contract.
- **Self-Destruct vs. Deactivate:** * The `selfdestruct` operation removes a contract's code/storage and sends remaining Ether to a target. However, this is dangerous; if someone sends Ether to a destroyed contract, the funds are lost forever.
 - **Best Practice:** Instead of destroying contracts, *deactivate* them by changing an internal state variable (e.g., `isActive = false`) that causes all function calls to revert, protecting user funds.

Section 2: Solidity Fundamentals & Data Structures

In this section, you will learn the fundamental syntax of the Solidity programming language, how to manage state, and how to define complex data structures for your smart contracts.

Solidity Data Types:

Solidity has a rich collection of built-in data types. Because gas optimization is a priority, selecting the correct data size is crucial.

- **Boolean:** Represents `true` or `false`. Supports standard logical operators:
- `!` (negation), `&&` (and), `||` (or), `==` (equality), and `!=` (inequality).
- **Integer: `uint` (Unsigned Integer):** Holds *only positive* values. Sizes range from `uint8` to `uint256` in steps of 8. For example, `uint8` stores values from 0 to 255. `uint` is an alias for `uint256`.

```
uint8 num1 = 127;
uint256 num2 = 6532195;
```

- **`int` (Signed Integer):** Holds *both negative and positive* values. Sizes range from `int8` (from -128 to 127) to `int256`.

```
int8 number1 = -127;
int256 number2 = 6532195;
```

- **String Literals:** Written with either double or single quotes (e.g., `"John Doe"`). They support escape characters like `\n` (newline).

```
string location = "66, mohakhali, BRAC University, Dhaka";
```

- **Address:** A unique type in Solidity that holds a 20-byte Ethereum account address. Address types have built-in members (properties/functions). For example, `.balance` queries the Ether balance of an address:

```
address owner = 0xc0ffee254729296a45a3885639AC7E10F9d54979;
```

For example, the condition below executes some code if the balance of the owner account is greater than or equal to 10.

NOTE: In Solidity, **members** refer to the properties (variables) and functions that belong to a type. Think of them as the built-in capabilities that come with certain data types.

```
if (owner.balance >= 10) {
    // sample code
}
```

- **Function Type:** From the contract's perspective, functions can be classified as internal and external. From the access modifiers perspective, functions are:
 - **Internal:** can only be called inside the current contract
 - **External:** consists of an address and a function signature and can be passed via or returned from external function calls
 - **public:** part of the contract interface and can be called either internally or via message calls
 - **private:** only visible inside the contract they are defined.
 - **view:** will read the state variable but will not modify it.
 - **pure:** can not read state variables and also cannot change them.
 - **payable:** allows functions to send or receive ethers.

A general syntax of the function is given below:

```
function name (<parameter types>) [internal|external|public|private] [pure|view|payable] [returns (<return types>)]
```

Sample 1:

A sample function called **retrieve**, which reads(view) the state variable **number** and returns a uint256 value **newNumber** are shown below:

```
function retrieve() public view returns (uint256) {
    uint256 newNumber = number + 100;
    return newNumber;
}
```

Sample 2: The sample function doesn't read or modify the state variable. It takes two parameters, called **firstName** and **lastName**, and returns the **fullName**.

```
function mergeName( string memory firstName, string memory lastName ) public pure returns (string memory){
    string memory fullName = string.concat(firstName, lastName);
    return fullName;
}
```

Sample 3:

The sample function modifies the state variable **number** and returns nothing.

```
function store(uint256 num) public {
    number = num;
}
```

- **Enum:** A user-defined type that restricts a variable to a predefined set of values. This increases code readability and reduces bugs. Enums store data as **uint8** indexes behind the scenes (e.g., Mobile = 0, Laptop = 1). For example, from the code in fig-1, we see a code that will not allow users to select any device other than a laptop, computer, or mobile. If you notice the setter functions, for example, **setMobile()**, we can select the mobile by using **DeviceList.Mobile** or **DeviceList(0)**. Both give the same result. Basically, enums stores user defined data that holds values like an array index in uint8. For example, the first member of the enum of the given code in fig-1 holds the value: Mobile = 0, the second one holds Laptop= 1, and the third one, Desktop = 2.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract EnumSample {
    enum DeviceList{ Mobile, Laptop, Desktop }
    DeviceList myDevice;

    function setMobile() public {
        myDevice = DeviceList.Mobile;
        // below line of code works same as the line of code above
        // myDevice = DeviceList(0)
    }

    function setLaptop() public {
        myDevice = DeviceList(1);
        // below line of code works same as the line of code above
        //myDevice = DeviceList.Laptop
    }

    function setDesktop() public {
        myDevice = DeviceList(2);
        // below line of code works same as the line of code above
        //myDevice = DeviceList.Desktop
    }

    // get state variable
    function getMyDevice() public view returns (DeviceList) {
        return myDevice;
    }
}
```

Fig-1: enum

- **Array:** Can be of fixed size **T[k]** or dynamic size **T[]**. Arrays have built-in members like **.push()** (adds to the end), **.pop()** (removes from the end), and **.length**. A sample code of arrays is given in Figure 2. **push**, **pop**, and **length** are some heavily used members of the array. However, fixed-size arrays only have access to their length among themselves.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

pragma solidity >=0.7.0 <0.9.0;

contract SampleVariables {
    string[3] fixedArray = ["Laptop", "Mobile", "Desktop"];
    string[] dynamicArray;
    // If you want to declare dynamic array with some initial length you can use:
    //string[] dynamicArray = new string[](3);
    // above line creates dynamic array with initial length 3

    function add(string memory val1, string memory val2) public {
        dynamicArray.push(val1);
        dynamicArray.push(val2);
        dynamicArray.pop();
        // fixedArray.push(val1); //==> This gives error
        // fixedArray.pop(); //==> This gives error
    }
    function getFixedArray() public view returns (string[3] memory) {
        return fixedArray;
    }
    function getDynamicArray() public view returns (string[] memory) {
        return dynamicArray;
    }
}
```

Fig-2: fixed and dynamic length arrays

- **Mapping:** Heavily used for large datasets, storing data in a **keyType => valueType** format. **keyType** can be any elementary type (like **address** or **uint**), but not complex types like structs or other mappings.

Sample 1: A sample mapping that holds addresses and their respective balance

```
mapping(address => uint) public balances;
```

Sample 2: A sample smart contract that contains a mapping of the vote count of individual candidates.

```
pragma solidity >=0.7.0 <0.9.0;

contract SampleVariables {
    mapping( string => uint256 ) public voteCount;

    function vote( string memory voterName ) public{
        voteCount[ voterName ] = voteCount[ voterName ] + 1;
    }
    function getVote( string memory voterName ) public view returns(uint256){
        return voteCount[ voterName ];
    }
}
```

- **Struct:** Allows you to define custom data types that group multiple variables together, similar to classes or objects in other languages. For example:

```

contract SampleVariables {
    struct Student {
        string name;
        uint256 id;
        bool isCurrentStatus;
    }
    Student[] studentList;

    function addStudent(string memory _name, uint256 _id, bool currentStatus) public {
        Student memory newStudent = Student(_name, _id, currentStatus);
        // you can also write it as mentioned below:
        // Student memory newStudent = Student({ name: _name, id: _id, isCurrentStatus: currentStatus });
        // also can be written like below:
        // Student memory newStudent;
        // newStudent.name = _name;
        // newStudent.id = _id;
        // newStudent.isCurrentStatus = currentStatus;
        studentList.push( newStudent );
    }

    function getStudents() public view returns(Student[] memory){
        return studentList;
    }
}

```

Fig-3: struct structure

- **Control structure:** The majority of the JavaScript control structures are provided in Solidity as well, with the exception of **goto** and **switch**. All with the normal semantics that are known from JavaScript or C Parentheses, for conditionals, must *not* be omitted; however, curly braces are possible to omit around bodies that are single-statement. Solidity provides these control structures:
 - **If**
 - **while**
 - **else**
 - **for**
 - **do-while**
 - **continue**
 - **return**
 - **break**

Solidity Contract:

- **License Identifier:** A smart contract starts with its license identifier in a comment which describes its copyright. If you do not want to specify a license, you can use **UNLICENSED** as the license identifier. For example: below is the syntax of mapping which holds the balance of addresses.
- **pragma:** We must mention the compiler version you are going to use to compile the contract under the “pragma” keyword. In the given sample contract in Fig-5, you will see the pragma directive telling us that the mentioned smart contract is written for solidity compiler version 0.7.0 or newer versions that does not include or exceed version 0.9.0. We also can use caret(^) to specify the versions. For example, ^0.7.0 tells us that the smart contract cannot be compiled using earlier than version 0.7.0 and also it will not work on a compiler starting from version 0.8.0

- **Comments:** Like in most other programming languages, comments can be used for providing hints throughout the code to make it easier to read. Solidity also features the *natspec* comment. A sample of *natspec* comments is provided in fig-4.

```

/// @author The Solidity Team
/// @title A simple storage example

contract SimpleStorage {

    uint storedData;

    /// Store 'x'.
    /// @param x the new value to store
    /// @dev stores the number in the state variable 'storedData'

    function set(uint x) public {

        storedData = x;

    }
}

```

Fig-4: sample natspec comments

- **Contract Structure:** Solidity contracts are much like **classes** in object-oriented programming languages. It can **inherit** elements from other contracts. Every contract contains declarations of elements such as functions, functions modifiers, state variables, struct, enum types, events etc. as needed for its purpose. A sample solidity program of smart contract is given below:

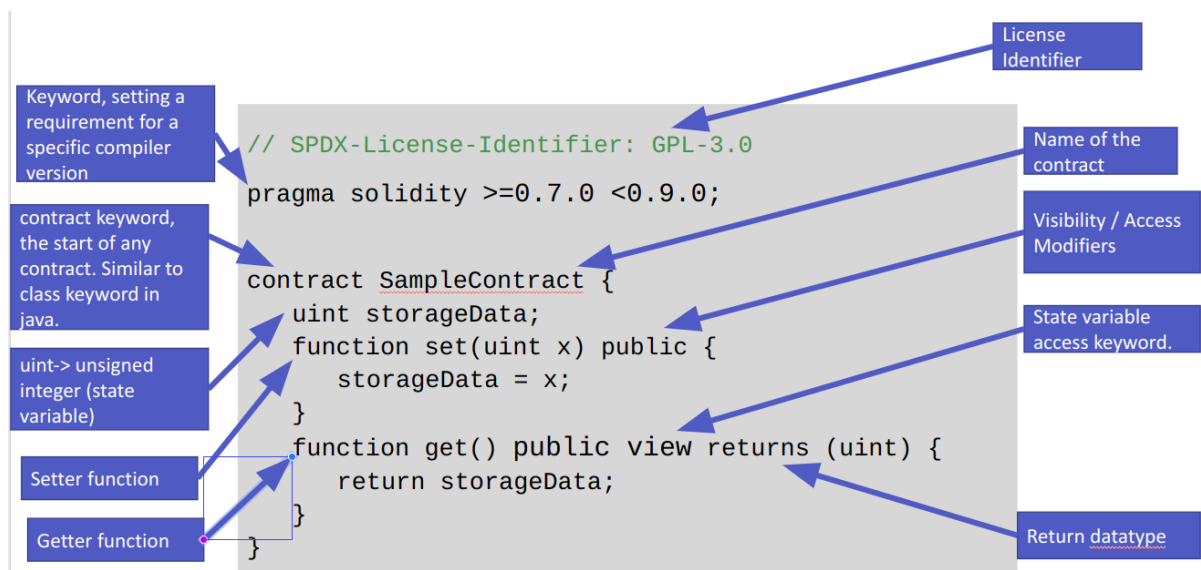


Fig-5: a simple smart contract

- **Function:** In solidity, function visibility varies to other contracts and can be called both internally and externally. There are four types of function visibility which are: **public**, **private**, **internal** and **external**. The visibility specifier is given after the type **for state variables** or in function, between parameter list and return parameter list.

```
function get() public view returns (uint) {  
    return storageData;  
}
```

For example, if we notice the above `get()` function, we will see a keyword called **public**. This declares the function is part of the contract interface and can be called either internally or via message calls.

A function's visibility determines who can call it, and its mutability dictates how it interacts with the blockchain state.

- **Visibility Modifiers:**

- **public:** Accessible internally and via external message calls.
- **private:** Only visible inside the contract where they are defined.
- **internal:** Callable inside the current contract and derived (inherited) contracts.
- **external:** Can only be called from outside the contract.

- **State Mutability:**

- **view:** Reads state variables but does not modify them.
- **pure:** Neither reads nor modifies state variables.
- **payable:** Allows the function to receive Ether (`msg.value`).

```
pragma solidity >=0.4.22 <0.6.0;  
  
contract Purchase {  
    address public seller;  
  
    modifier onlySeller() { // Modifier  
        require( msg.sender == seller, "Only seller can call this." ); _;  
    }  
  
    function abort() public view onlySeller {  
        // Modifier usage // ...  
    }  
}
```

- **State vs. Local Variables:**

- **State Variables:** Permanently stored in contract storage (costs gas).
- **Local Variables:** Initiated when a function is called and destroyed upon completion (stored in the stack/memory, costing no/low gas). When using reference types like **string** locally, you must specify the **memory** keyword. For example, in the fig-7, **uint statePrice** is a state variable.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

contract SampleVariables {
    uint statePrice = 100;

    // adding price worth of statePrice added with vat to state variable
    function reducePrice(uint vat) public {
        uint localPrice = 50;

        localPrice += vat;
        statePrice = localPrice;
    }

    // function that return a local variable data
    function checkLocalName() public pure returns (string memory) {
        string memory localText = "CSE446-Blockchain & Cryptocurrencies";

        return localText;
    }

    // get state variable
    function get() public view returns (uint) {
        return statePrice;
    }
}
```

Fig-7: state and local variables

On the other hand, in fig. 7, **localPrice** is a local variable that is initiated when the function is called and gets destroyed when the call is completed. Local variables get stored in the stack and do not cost any gas. However, there are certain types of data that by default get stored in storage. **String** is one of them. From fig. 7, we see a string variable called **localText**. Therefore, to use string type data inside a function like a local variable, we must use a memory keyword that declares that the data is neither stored in storage nor in the stack. It gets stored in memory and costs less gas than storage.

- **Events:** allow the convenient use of EVM logging facilities. DApp frontends (using JavaScript/Web3.js) can listen to these events to update the user interface dynamically without constantly querying the blockchain. An event is dispatched using the **emit** keyword. A sample contract of events is shown in Fig. 8.

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract SampleEvent {
    event Deposit( address indexed _from, uint indexed _id, uint _value );

    function deposit(uint _id) public payable {
        // Events are emitted using `emit`, followed by
        // the name of the event and the arguments
        // (if any) in parentheses. Any such invocation
        // (even deeply nested) can be detected from
        // the JavaScript API by filtering for `Deposit`.
        emit Deposit(msg.sender, _id, msg.value);
    }
}
```

Fig-8: event and emit

3 Reference Type & Data Area

Like other programming languages, Solidity has different types of **reference-typed data**, such as **structs**, **arrays**, and **mappings**. These data types must be handled carefully because they **pass the reference** (the memory address) instead of the actual value.

Because of how the Ethereum Virtual Machine (EVM) operates, reference-typed data requires you to explicitly specify the **Data Area** (the location where the type is stored) during declaration.

Question to explore at home: We need to explicitly mention the data area for reference typed data. But if we use these reference-typed data as state variables, we don't need to mention them explicitly. Why?

Please go through this [documentation](#) to know more about the data area.

Example of reference type and data area:

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract Draft {

    uint[] numList = [ 10, 20, 30, 40 ];

    function add( uint _num ) public { 24683 gas
        /* the line below is passing the reference instead of value
        * the data area can be either memory or storage */
        uint[] storage someValue = numList;
        // QUESTION TO EXPLORE: what happens if we use memory instead of storage in the above line

        // The assignment below will modify the original array: numList
        someValue[3] = _num;
    }

    /*
    ## QUESTION TO EXPLORE: can we use param as "uint256[] calldata _sampleArray" instead
    of using "uint256[] memory _sampleArray" ? Explain why
    */
    function modifyArray(uint256[] memory _sampleArray) public pure returns (uint256[] memory) {
        for (uint256 i = 0; i < _sampleArray.length; i++) {
            _sampleArray[i] *= 2; // Example modification: double each element
        }
        return _sampleArray;
    }

    // check the current value of numList
    function getValue() public view returns ( uint[] memory ){ infinite gas
        return numList;
    }
}
```

Figure - 1

The example above, within the **add** function, we updated **someValue**, however because of the pass-by reference type, **numList** has been modified. The

modifyArray function receives a reference type `_sampleArray` of **memory** data area. Because its lifespan is limited to function calls, and we also modified it within the function.

4 Error Handling

In Solidity, error handling is fundamentally different from traditional programming languages. Instead of catching exceptions and continuing execution, Solidity uses **state-reverting exceptions**. When an error occurs, the Ethereum Virtual Machine (EVM) stops execution, reverts all changes made to the state during that transaction, and refunds any remaining gas to the caller.

While there are multiple ways to handle errors, we will focus on the three primary mechanisms: **require**, **revert**, and **assert**.

4.1 Require

require is the most commonly used function for validating conditions in smart contracts. It is ideal for validating user inputs, checking conditions before execution, or verifying responses from external contracts.

- **Use Case:** "Is the user allowed to do this?" or "Is the input valid?"
- **Syntax Variants:**
 - `require(condition);`
 - `require(condition, "Error Message");` (Recommended)
 - `require(condition, CustomError());` (Advanced/Gas efficient)

4.2 Revert

Conceptually, both `revert()` and `require()` do the exact same thing: they handle errors by reverting the transaction. However, `revert()` is preferred when the condition to check is complex or deeply nested inside `if/else` statements.

- **Use Case:** "If this complex scenario happens, abort the transaction."
- **Syntax:** `revert("Error Message");`

4.3 Assert

`assert()` is used to test for internal logical errors and check invariants (conditions that should *never* be false if your code is logically sound). Properly functioning contracts should never reach a failing `assert` statement.

- **Use Case:** "Check if my math is broken" or "Ensure the contract state isn't corrupted."
- **Syntax:** `assert(condition);`

```

contract ErrorHandling {
    uint256 totalTransactionToday = 0;

    // sending some amount of ETH to a _receipiant address
    function sendCurrency(address payable _receipiant) payable public {
        // require(): if condition false, throws error and reject the transaction
        require(msg.value < 10, "Condition is not true, So it will throw error"); // msg.value is the mount of ETH sender sending
        // if the condition above is passed, the code below will be executed

        // revert(): throws error and the transaction will be reverted. However, it is used within a condition
        if(totalTransactionToday >= 2) {
            revert("Your transaction limit for today has been exceeded");
        }

        // assert(): revert the transaction if the condition is false
        assert(msg.sender.balance - msg.value >= 0); // msg.sender.balance is the available balance of sender
        _receipiant.transfer(msg.value);
        totalTransactionToday++;
    }
}

```

Figure 5: require, revert, assert

5 Members of Different Types

In Solidity, "members" are various elements (functions or properties) natively associated with different data types. This is similar to the built-in methods or attributes of objects in other programming languages (e.g., `array.append()` in Python).

Below are the most critical members you will need when developing your smart contracts, especially for the final lab project.

5.1 Array Members

Arrays in Solidity come with several built-in methods to manage their elements and size:

- **push(x)**: Adds an element `x` to the end of a dynamic array. (e.g., `myArr.push(5);`).
- **pop()**: Removes the last element from the end of a dynamic array. (e.g., `myArr.pop();`).
- **length**: Returns the current number of elements in the array. (e.g., `uint size = myArr.length;`).
- **delete**: Deletes the value at a specific index or clears the whole array.
 - *Important Note*: **delete** does not change the array's length. Instead, it resets the element to its default initialization value (e.g., `0` for `uint`, `false` for `bool`, and `""` for `string`).
 - Example: `delete myArr[1];` resets the second element to default. `delete myArr;` resets all elements.

5.2 String Members

As discussed previously, strings have very limited native members.

- **string.concat(str1, str2)**: Concatenates two strings together. (*Available in Solidity 0.8.12 and later*).

5.3 Address Members

The `address` type has specific members used for checking balances and moving Ether.

- `some_address.balance`: Returns the current Ether balance of the address (in wei).
- `payable(some_address).transfer(amount)`: Sends a specified `amount` of wei to the address. Reverts the transaction if it fails (e.g., if the contract doesn't have enough balance).

5.4 The Global `msg` Object

The `msg` object is a special global variable that contains properties related to the current blockchain transaction or message call.

- `msg.sender`: The address of the entity (user or another contract) that is currently calling the function. **(Crucial for access control)**.
- `msg.value`: The amount of Ether (in wei) sent along with the current message call. **(Crucial for payable functions and Escrow logic)**.
- `msg.data`: The complete, raw calldata of the current function call.
- `gasleft()`: Returns the remaining gas for the current transaction. *(Note: In older versions of Solidity, this was known as `msg.gas`, but that has been deprecated in modern versions).*

6 Bytes & Strings

In Solidity, `bytes` is a heavily used reference type. It is a special type of array that holds a sequence of bytes.

- **Fixed-size arrays**: To store fixed-size arrays of bytes, `bytes1`, `bytes2`, `bytes3`, ..., up to `bytes32` are used. For example, `bytes1` exactly holds 1 byte, `bytes2` exactly holds 2 bytes, and so on.
- **Dynamic byte arrays**: If we only use `bytes` (excluding the numbers), it becomes a dynamic byte array that is resizable.
- **Array of dynamic bytes**: Additionally, `bytes[]` is an array where each element is itself a dynamic byte array.

```

contract BytesExamples {
    // Fixed-size byte arrays
    bytes1 private byte1Data; // hold 1 byte == 1 character
    bytes2 private byte2Data; // hold 2 byte == 2 character
    bytes3 private byte3Data; // hold 3 byte == 3 character
    bytes32 private byte32Data; // hold 32 byte == 32 character

    // Dynamic byte array: hold dynamic amount of byte == dynamic amount of character
    bytes private dynamicByteArray;

    // Array of dynamic byte arrays: hold dynamic amount of bytes array
    bytes[] private arrayOfByteArray;

    constructor() {
        // infinite gas 289200 gas
        byte1Data = 0x41; // Stores the Hex ASCII value for 'A'
        // byte2Data = "AB"; // We can assign in string format but ASCII value will get stored | 0x4142 is = AB
        // byte2Data = "ABC"; // This will give error since 2 byte can be stored
        // byte2Data = "A"; // This will store 0x4100 | For A, 0x41 will be stored and an empty character 00 is stored
        // byte2Data = 0x41; // But this will give error | if we assign Hex value then we must EXACTLY 2 byte Hex here
        byte3Data = 0x414243; // Stores the Hex ASCII values for 'ABC'
        byte32Data = "abcdefghijklmnopqrstuvwxyz123456"; // Stores the ASCII of 32 character

        dynamicByteArray = new bytes(5); // Creates a dynamic byte array of length 5
        // dynamicByteArray[0] = 'H';
        // dynamicByteArray[1] = 'e';
        // dynamicByteArray[2] = 'l';
        // dynamicByteArray[3] = 'l';
        // dynamicByteArray[4] = 'o';
        // similarly we can store directly like below
        dynamicByteArray = "Hello";

        arrayOfByteArray.push("Hello"); // Adds a new dynamic byte array with "Hello" that has length 5
    }

    // returns ASCII Code in Hex format
    // Explore yourself at home: why bytes1, bytes2 bytes3 doesn't require memory/calldata keyword below ?
    function getFixedByteArray() public view returns (bytes1, bytes2, bytes3, bytes32) {
        // 9280 gas
        return (byte1Data, byte2Data, byte3Data, byte32Data);
    }

    // returns ASCII Code in Hex format
    function getDynamicArrays() public view returns (bytes memory, bytes[] memory) {
        // infinite gas
        return (dynamicByteArray, arrayOfByteArray);
    }
}

```

Figure 6: variants of bytes

Question to explore at home: What happens if we write `dynamicByteArray = "HelloA"` and if we write `dynamicByteArray[5] = "A"`, in the above code?

String

Working with strings is a bit challenging in Solidity as it does not have any members or built-in methods to manipulate them, except for `string.concat()`.

String is unique in Solidity compared to other well-known programming languages. Under the hood, a `string` is essentially a special typed array and is equal to `bytes`. However, it **does not** have the access capabilities (like index access or `.length`) that an array or `bytes` type has.

Direct modification or accessing specific characters is not available natively. This limitation is due to the design goals of Solidity, EVM constraints, and strict gas efficiency. We can, however, manipulate strings by converting them into `bytes`, modifying the bytes, and casting them back into a `string`.

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.8.2 <0.9.0;

contract StringManipulation {

    bytes private dynamicByteArray;

    // cast string into Hex ASCII and store
    function storeString(string memory _str) public {    infinite gas
        dynamicByteArray = bytes(_str);
    }
    // returns ASCII Code in Hex format
    function getRawStringData() public view returns( bytes memory){    infinite gas
        return dynamicByteArray;
    }
    // cast and return string
    function getString() public view returns( string memory){    infinite gas
        return string(dynamicByteArray);
    }
}
```

Figure 7: String & Byte Conversion

Question to explore at home: How can you compare two strings to determine whether they are equal or not? Explore at least two approaches to do it.

7 Function Calls - Internal & External

A. Internal Function Call

To call a function from within the same contract instance, you can simply call it by its name. However, to restrict a function so that it *cannot* be called from outside the contract (e.g., by a user or another contract), we use the **internal** keyword.

These are known as internal function calls. While public functions can also be called internally, explicitly marking helper functions as **internal** is a best practice for security and gas optimization.

```
contract InternalCall {
    function process(uint a, uint b) public pure returns (uint result) {
        if(a > 5){
            return deduct( a ,b );
        }
        else{
            return add(a, b);
        }
    }
    function add(uint a, uint b) internal pure returns (uint ret) {
        uint val = a + b;
        return val;
    }
    function deduct(uint a, uint b) public pure returns (uint ret) {
        uint val = a - b;
        return val;
    }
}
```

Figure - 2

Figure 2 shows two function calls, “add” and “deduct,” invoked from the function “process”. Though the two functions can be called from another function within the same contract, the difference is that “add” is an **internal** function and it is only used from another function call within the same contract or a derived contract. On the contrary, the “deduct” can also be called externally.

B. External Function Call

In Solidity, contracts can interact with one another. You can call a function from one contract inside another contract by instantiating a reference to it.

To define a function that is explicitly designed to be called by other contracts (or external users) but *not* internally, we use the **external** keyword. External functions are highly gas-efficient when handling large arrays of data because they read directly from **calldata**.

To call an external contract, you need two things:

1. The **Interface** or imported code of the target contract.
2. The **Deployed Address** of the target contract.

```

1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.8.2 <0.9.0;
4
5 contract ExternalParent {
6     uint256 public value;
7
8     // This is an external function
9     function setValue(uint256 _value) external { 22542 gas
10         value = _value;
11     }
12
13     // External function to get the value
14     function getValue() public view returns (uint256) { 2415 gas
15         return value;
16     }
17 }
18

```

Figure 3: Contract with external function

Finally, **callSetValue** and **callGetValue** can call the functions of the **ExternalParent** contract externally using simple dot notation. Notice, **setValue** is an **external** function, and **getValue** is public, and both can be called externally.

```

1 // SPDX-License-Identifier: GPL-3.0
2
3 pragma solidity >=0.8.2 <0.9.0;
4
5 import "../ExternalParent.sol";
6
7 contract ExternalChild {
8
9     // defining the externalContract object using instance of ExternalParent
10    // We can do this if we import the contract
11    ExternalParent externalContract;
12
13    // Setting the address of external contract ExternalParent
14    function setExternalContract(address _contractAddress) public { 24700 gas
15        externalContract = ExternalParent(_contractAddress);
16    }
17    // Alternatively you can use constructor to set external contract address at the time of deployment
18    // constructor(address _contractAddress) {
19    //     externalContract = ExternalParent(_contractAddress);
20    // }
21
22    // Call setValue of ExternalParent contract
23    function callSetValue(uint256 _value) public { infinite gas
24        externalContract.setValue(_value); // External function call
25    }
26
27    // Call the getValue of ExternalParent contract
28    function callGetValue() public view returns (uint256) { infinite gas
29        // The getValue function are also being called externally but it is a public function
30        return externalContract.getValue();
31    }
32 }
33

```

Figure 4: Calling external and public functions externally

8 Working with multiple contracts

In Solidity, complex Decentralized Applications are rarely written in a single file. We manage complexity by dividing logic across multiple smart contracts. You can interact with other contracts either by **inheriting** them (using Object-Oriented principles) or by **importing** them.

Note: In this specific lab, we will focus solely on importing, as inheritance introduces advanced OOP

import

When using **import** to interact with external contracts, you are essentially linking your contract to another deployed contract's interface.

- **Visibility Rules:** No **private** or **internal** data/functions can be directly accessed from the imported contract.
- To interact with the imported contract, the target functions must be marked as **public** or **external**.

Figure 8 shows an example of a parent contract titled "ImportSampleParent".

```
contract ImportSampleParent {  
    string private name = "This is Parent";  
  
    function updateName(string memory _newName) public { infinite gas  
        name = _newName;  
    }  
  
    function retrieveName() public view returns (string memory){ infinite gas  
        return name;  
    }  
}
```

Figure 8: Sample Contract that will get imported

This contract is externally imported in the contract "ImportSampleChild", which is presented in Figure 9 below.

```

pragma solidity >=0.8.2 <0.9.0;

// import contract
import "./ImportSampleParent.sol";

contract ImportSampleChild {

    // To use the imported/external contract we need to instantiate it with Contract name
    ImportSampleParent ParentContract;
    uint256 number = 5;

    // next we need to assign a contract address into ParentContract object.
    // we can do it using function of constructor | Both example are provided below
    // Setting the address of external contract
    function setExternalContractAddress(address _contractAddress) public { 24722 gas
        // the contract reference type ParentContract should be assign with an address casted by imported contract reference ImportSampleParent
        ParentContract = ImportSampleParent(_contractAddress);
    }

    // Alternatively you can use constructor to set external contract address at the time of deployment
    constructor(address _contractAddress) { infinite gas 295600 gas
        ParentContract = ImportSampleParent(_contractAddress);
    }

    /**
     * @dev Store value in variable
     * @param _newName value to store
     */
    function setName(string memory _newName) public { infinite gas
        ParentContract.updateName(_newName);
    }

    // fetching data from parent
    function retrieve() public view returns (string memory){ infinite gas
        return ParentContract.retrieveName();
    }
}

```

Figure 9: A contract importing and using another one

Some key things about Solidity

- We cannot return data from a function if it modifies the state variable and is being called from EOA accounts.
- In solidity, there are only a few built-in methods/members available.
- In solidity, string manipulation is challenging as it does not have built-in methods/members except for concat. We cannot even directly compare two strings. Though some packages/libraries can solve it in one line, we will work only on raw solidity in this course.
- Use the **view**-in function if you are reading data.
- Use **pure** in the function if you are neither reading nor modifying data from the contract state.
- While interacting with function parameters and return data type for reference typed data, we are required to mention data areas such as memory, calldata, and storage. Proper use of these data areas can save a lot of gas.
- A few error mechanisms and validation are handled in a bit different way in Solidity.
-

According to the Lab Finish the Lab Tasks: <https://forms.gle/mg8HLYBKkTYvL7rJ6>

References:

- Official Documentation: <https://docs.soliditylang.org/en/v0.8.26/>
- Smart Contract Best Practices: <https://github.com/ConsenSys/smart-contract-best-practices>
- Solidity Cheatsheet: <https://docs.soliditylang.org/en/latest/cheatsheet.html#members-of-bytes-and-string>
- <https://www.w3schools.io/blockchain/solidity-tutorials/>
- <https://www.tutorialspoint.com/solidity/index.htm>